

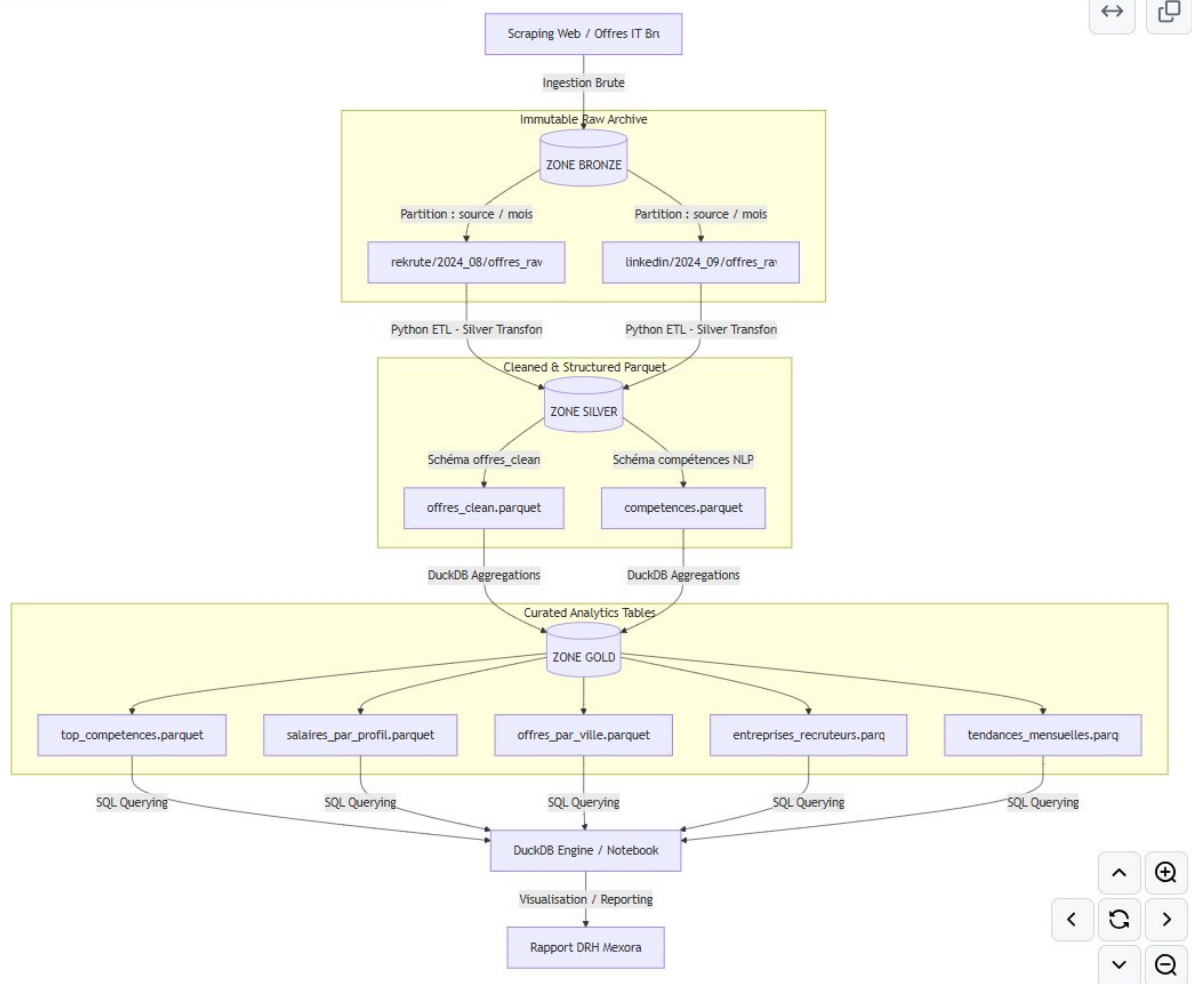
Pipeline de Traitement des Données IT

Mexora RH Intelligence — Conception d'un Data Lake pour l'Analyse du Marché de l'Emploi

1. Architecture du Data Lake

Le Data Lake de Mexora RH Intelligence est structuré en **trois zones logiques (Bronze, Silver, Gold)**. Cette organisation permet d'isoler l'acquisition des données, leur nettoyage et leur valorisation analytique.

1.1 Diagramme d'Architecture



1.2 Justification des Formats de Stockage

Zone	Format Choisi	Pourquoi ce format ?	Pourquoi pas les autres ?
Bronze	JSON	Préservation fidèle : Les API de scraping retournent du semi-structuré. Le format JSON permet d'archiver la structure imbriquée d'origine sans perte d'information (schéma flexible).	Le CSV gère mal les listes (ex: langue_requise) et les champs libres. Le Parquet imposerait un typage immédiat (Schema-on-Write) risquant de bloquer l'ingestion de données mal formées.
Silver	Parquet	Compression et performance : Le format colonnaire Parquet (avec compression Snappy) réduit l'espace disque de plus de 70% et accélère les lectures sur les colonnes cibles.	Le JSON n'a plus de valeur ici car les structures complexes ont été aplaties. Le CSV souffre de lenteur de parsing et d'absence de typage natif.
Gold	Parquet	Optimisation analytique : Les requêtes DuckDB agrègent des millions de lignes sur des colonnes spécifiques (ex: médiane des salaires) en quelques millisecondes.	Les bases relationnelles classiques (DWH) ou CSV ralentiraient significativement les requêtes analytiques interactives locales.

1.3 Choix et Concepts de Conception

1. Schema-on-Read vs Schema-on-Write

Schema-on-Write (Entrepôt classique) : Le schéma est défini avant le chargement. Toute donnée non conforme est rejetée à l'entrée (cas de la base PostgreSQL). Ce modèle garantit la qualité mais ralentit l'ingestion de nouvelles sources.

Schema-on-Read (Notre Data Lake) : Les données brutes sont chargées immédiatement (Bronze) sans schéma strict. Le schéma et les transformations ne sont appliqués que lors de la lecture ou de l'extraction (Silver/Gold). Cela garantit qu'aucune donnée n'est perdue à l'entrée, même si elle comporte des erreurs de format.

2. Pourquoi la zone Bronze doit être immuable

La zone Bronze représente la **version historique de vérité**. Si un bug est découvert dans l'algorithme de nettoyage de la zone Silver, nous pouvons corriger le pipeline Python et réexécuter le traitement à partir de la zone Bronze. Si la zone Bronze était modifiable, nous perdriions la capacité de retraiter les données historiques brutes en cas d'erreur ou d'évolution des règles métier.

3. Éviter un "Data Swamp" (Marécage de Données)

Un Data Lake devient un marécage lorsque les données y sont déposées sans gouvernance ni catalogue. Pour Mexora RH, nous évitons cela par :

- Une arborescence stricte et normalisée des répertoires.
- Une gestion automatisée des logs (enregistrant les dates et volumes à chaque étape).
- Un référentiel de compétences externe et d'entreprises assurant une jointure stricte.
- L'écriture systématique de métadonnées d'ingestion (provenance, horodatage) dans chaque partition Bronze.

4. Justification du Partitionnement

Bronze (par source / mois) : L'ingestion se fait de manière périodique (journalière ou mensuelle) et par scraper. Le partitionnement par source et par mois permet de charger de petits volumes rapidement, et d'isoler une source en cas de panne de son scraper.

Silver (par ville / mois) : La zone Silver sert à l'enrichissement. Le partitionnement par ville et par mois optimise les requêtes d'analyse géographique qui constituent la majorité des requêtes métiers (ex: filtrer les offres sur Tanger pour le mois en cours sans scanner tout le dataset national).

2. Transformations et Règles de Nettoyage (Bronze → Silver)

Le pipeline Python applique les transformations avancées suivantes lors du passage à la zone Silver :

2.1 Normalisation Géographique

Règle : Les graphies incorrectes ou abrégées des villes marocaines (ex: "casa", "CASABLANCA", "Tangier") sont mappées vers un référentiel standardisé à l'aide d'expressions régulières insensibles à la casse.

Enrichissement : Jointure avec un dictionnaire géographique pour ajouter la colonne *region_admin* (ex: Casablanca → Casablanca-Settat), permettant des analyses par régions administratives nationales.

2.2 Standardisation des Contrats

Règle : Les types de contrats multiples ("cdi", "Contrat à durée indéterminée", "Permanent") sont unifiés sous une nomenclature propre à **5 modalités** : CDI, CDD, Freelance, Stage, ANAPEC. Les modalités inconnues sont classées sous "Autre".

2.3 Typage de l'Expérience

Règle : Extraction numérique des années d'expérience à partir de chaînes libres (ex: "3-5 ans" → *experience_min_ans* = 3, *experience_max_ans* = 5). Les mentions comme "Débutant" ou "Stage" sont converties en (0, 2) ans. Les profils "Senior" ou "Confirmé" sans chiffre précis sont mappés par défaut à partir de 5 ans minimum.

2.4 Conversion et Parsing des Salaires

Règle :

- Extraction des nombres depuis la chaîne *salaire_brut* (ex: "15K-20K MAD", "15000dh").
- Conversion des montants exprimés en milliers ("K") en valeurs absolues.
- Conversion des montants en Euros (€) vers le Dirham Marocain (MAD) : taux de change moyen **1 EUR = 10.8 MAD**.
- Pour les fourchettes, calcul du salaire médian.
- Filtre de cohérence : salaires < 3 000 MAD ou > 100 000 MAD/mois marqués comme erronés (*salaire_connu* = False).

2.5 Cohérence des Dates

Règle : Les dates de publication et d'expiration sont converties en type *datetime*. Si *date_publication* > *date_expiration*, un flag *date_coherence_ok* est positionné à **False** pour identifier l'anomalie sans rejeter la ligne (Schema-on-Read). Les colonnes *annee* et *mois* de publication sont extraites.

2.6 Extraction NLP des Compétences IT

Méthodologie :

- Chargement du référentiel JSON contenant **300 compétences IT** organisées en 8 familles technologiques.
- Fusion des champs `competences_brut` et `description` convertis en minuscules.
- Nettoyage des ponctuations et caractères spéciaux.
- Recherche par expressions régulières (Regex) en respectant les frontières de mots (\b) pour éviter les faux positifs (ex : "node" ne doit pas matcher sur "méthode").
- Tri des alias par ordre de longueur décroissante pour prioriser les expressions complexes (ex : matcher "node.js" avant "node").
- Génération d'une table relationnelle aplatie associant chaque offre à ses compétences détectées.

3. Rapport Statistique du Traitement

L'exécution du pipeline complet sur le jeu de données généré de **5 000 offres** produit les statistiques suivantes :

Métrique	Valeur Avant (Bronze)	Valeur Après (Silver)
Volume total d'offres	5 000	5 000
Variété des noms de villes	15 valeurs distinctes	5 valeurs normalisées
Variété des types de contrats	11 formes libres	5 types standards
Intitulés de poste distincts	48 dénominations	10 familles de profils IT unifiées
Salaires exploitables (%)	68.1% (bruts non typés)	68.1% (propres en MAD)
Offres avec compétences détectées (%)	N/A	95.8% (au moins 1 compétence)
Nombre de dates incohérentes (pub > exp)	N/A	244 offres signalées

Cas Limites Rencontrés et Résolus

Les symboles dans les compétences (ex : C#, .NET) : Les expressions régulières standards basées sur \b excluent les caractères spéciaux comme # ou .. Nous avons ajusté la regex NLP pour ces alias spécifiques en utilisant des expressions alternatives : `(?:^|\s)c#(?:\s|$|,|\. )`.

Les salaires en "K" : Les scrapings contiennent des notations comme 15K. Le script détecte le "K" et multiplie par 1 000 le montant avant d'évaluer la fourchette.

Le bruit textuel dans les descriptions : L'extraction se limitant au simple matching d'alias, certains mots courants pourraient matcher. Nous avons réduit le référentiel d'alias en éliminant les synonymes trop courts ou ambigus (ex: "r" pour le langage R doit être recherché de manière extrêmement restrictive).